



دانشگاه صنعتی امیرکبیر
دانشکده مهندسی کامپیوتر



«مبانی و کاربردهای هوش مصنوعی ترم بهار ۱۴۰۵»

پروژه اول

جستجوی آگاهانه و ناآگاهانه و جستجوی تخصصی

در انجام پروژه‌ها به نکات زیر توجه فرمائید:

- ۱- زبان برنامه‌نویسی برای انجام این پروژه، تنها می‌تواند پایتون باشد.
- ۲- مطابق قوانین دانشگاه هر نوع کپی‌برداری و اشتراک کار دانشجویان غیرمجاز است و انجام پروژه‌ها باید به صورت انفرادی انجام شود؛ در صورت مشاهده چنین مواردی با طرفین شدیداً برخورد خواهد شد.
- ۳- توجه نمایید که بخشی از نمره اصلی شما به گزارشکاری که طبق موارد خواسته‌شده برای این پروژه نگارش می‌کنید اختصاص دارد
- ۴- در صورت هر گونه سوال و یا ابهام از طریق ایمیل زیر می‌توانید در ارتباط باشید:
h.masroor@aut.ac.ir
- ۵- توجه نمائید فایل پاسخ پروژه‌ها تنها در صورت آپلود در سامانه کورسز پذیرفته خواهد شد و ارسال از طریق ایمیل و یا روش‌های دیگر بررسی نخواهد شد.
- ۶- فایل پروژه خود را با فرمت StudentID_P01.zip تا ساعت ۲۳:۵۹ روز ۱۴۰۵/۰۴/۲۴ فقط در بخش مربوطه در سایت درس آپلود نمائید.

"فهرست مطالب پروژه"

۲.....	مقدمه
۳.....	بخش اول: الگوریتم‌های جستجوی آگاهانه و ناآگاهانه
۳.....	هدف کلی پروژه
۳.....	ساختار کلی پروژه
۴.....	دنیای بازی: تالار زمانی کرونوس
۷.....	تعریف دقیق state
۷.....	ماموریت شما
۸.....	الگوریتم‌هایی که باید پیاده‌سازی شوند
۱۰.....	خروجی مورد انتظار الگوریتم‌ها
۱۱.....	ضوابط ارزیابی
۱۲.....	بخش دوم: الگوریتم‌های جستجوی خصمانه
۱۲.....	اکشن‌های نبرد
۱۳.....	الگوریتم‌هایی که باید پیاده‌سازی شوند
۱۴.....	ضوابط ارزیابی
۱۵.....	گزارش پروژه و الزامات

مقدمه‌ی داستانی

در جایی بیرون از فضا و زمان، جایی در بین ثانیه هایی که هیچوقت اتفاق نیفتادند، تالاری وجود دارد به نام تالار زمانی کرونوس. این تالار شبیه یک آزمایشگاه معمولی نیست دیوارهایش از خاطراتی که هیچوقت وجود نداشتند ساخته شده اند و کف آن روی خطوط زمانی شناور است، و در مرکز این تالار چیزی می‌درخشد که تمام جهان ها به آن متکی هستند: Chronos Core، قلب زمان.

برای سال‌های طولانی، Chronos Core آرام کار می‌کرد. هر تصمیم، هر مسیر، هر اتفاق کوچک، در تار و پود زمان دوخته می‌شد. اما حالا چیزی اشتباه شده است. شکاف‌هایی در زمان باز شده‌اند، قطعه‌های انرژی زمانی در سراسر نقشه پخش شده‌اند، و موجوداتی از آشوب، بین لحظه‌ها پرسه می‌زنند. شما در نقش یک عامل کوچک اما شجاع به نام Chronos Agent وارد بازی می‌شوید. یک ربات زمان‌بان که مأموریت دارد قبل از فروپاشی کامل خط زمانی، تمام قطعه‌های گمشده را جمع کند، شکاف‌ها را ببندد، و خودش را به Chronos Core برساند. این پروژه از نظر حال‌وهوا یک ادای احترام کوچک و دوستانه به فصل دوم Loki است. نه بازسازی آن، نه استفاده‌ی مستقیم از داستانش، بلکه یک ادای احترام به ایده‌هایی مثل بافت زمان، شاخه‌های زمانی، فداکاری برای حفظ چندجهانی، و این سؤال جذاب که: اگر آینده هنوز نوشته نشده باشد، بهترین مسیر را چطور پیدا می‌کنیم؟

اینجا دقیقا همان جایی است که هوش مصنوعی وارد می‌شود.

بخش اول: الگوریتم های جستجوی آگاهانه و ناآگاهانه

هدف کلی پروژه

در این پروژه، بازی و محیط گرافیکی از قبل آماده شده است. شما لازم نیست بازی را از صفر بسازید. کار اصلی شما این است که به Chronos Agent مغز بدهید.

Chronos Agent باید بتواند در یک نقشه‌ی شبکه‌ای حرکت کند، از موانع عبور نکند، از دشمنان زمانی دوری کند، قطعه‌های زمان را جمع کند، شکاف‌ها را ببندد، و در نهایت به هسته‌ی زمان برسد. بازی شامل الگوریتم‌هایی مثل BFS، DFS، UCS، Greedy، و A^* است و رابط گرافیکی برای اجرای آن‌ها آماده شده است. دکمه‌های این الگوریتم‌ها در نسخه‌ی آماده‌ی بازی وجود دارند و دانشجو باید منطق پشت آن‌ها را کامل کند.

ساختار کلی پروژه

این تمرین شامل دو بخش با نام‌های Chronos Duel و Chronos Weaver است. در هر بخش دو فایل ارائه شده است. فایل‌های `chronos_duel.py` و `chronos_weaver.py` یک پیاده‌سازی پایه از بازی‌ها را ارائه می‌دهند تا ساختار کلی برنامه، کلاس‌ها و منطق اجرا مشخص باشد. این پیاده‌سازی‌ها صرفاً به عنوان نقطه شروع در نظر گرفته شده‌اند.

در مقابل، فایل‌های `chronos_duel_todo.py` و `chronos_weaver_todo.py` نسخه‌های تمرینی هستند که در آن‌ها بخش‌های مهمی از منطق برنامه خالی گذاشته شده است. هدف این تمرین این است که با تکمیل این بخش‌ها، یک پیاده‌سازی بهتر و هوشمندتر نسبت به نسخه پایه ایجاد کنید. بنابراین می‌توانید از نسخه‌های پایه برای درک ساختار برنامه استفاده کنید، اما انتظار می‌رود راه‌حل نهایی شما از نظر الگوریتم و کیفیت تصمیم‌گیری بهتر از آن‌ها باشد.

دنیای بازی: تالار زمانی کرونوس

نقشه‌ی بازی یک صفحه‌ی خانه‌به‌خانه است. هر خانه یک معنی خاص دارد. بعضی خانه‌ها امن‌اند، بعضی خطرناک، بعضی میانبر زمانی هستند، و بعضی بخشی از مأموریت اصلی شما را تشکیل می‌دهند.

۱- شخصیت اصلی: Chronos Agent

Chronos Agent همان ربات کوچک آبی‌رنگی است که روی نقشه حرکت می‌کند. او سریع، بی‌صدا و کمی بیش از حد شجاع است. وظیفه‌ی او این نیست که دشمنان را شکست دهد؛ وظیفه‌ی او این است که بهترین مسیر ممکن را پیدا کند.



او می‌تواند به چهار جهت اصلی حرکت کند و گاهی می‌تواند صبر کند چون در یک دنیای زمانی، گاهی بهترین حرکت این است که هیچ حرکتی نکنی.

۲- Chronos Core

Chronos Core مقصد نهایی شماست. تا وقتی همه‌ی قطعه‌های زمانی جمع نشده‌اند و همه‌ی شکاف‌های لازم بسته نشده‌اند، رسیدن به هسته کافی نیست. Chronos Core فقط زمانی پایدار می‌شود که همه‌ی پیش‌نیازهای زمانی کامل شده باشند. در بازی هم شرط برد این است که عامل روی خانه‌ی Core باشد، همه‌ی Fragmentها جمع شده باشند و همه‌ی Riftها بسته شده باشند.



۳- Fragments یا قطعه‌های زمان

Fragmentها خرده‌زمان‌هایی هستند که از Chronos Core جدا شده‌اند. روی نقشه با نمادهای درخشان دیده می‌شوند. دانشجو باید الگوریتمی پیاده‌سازی کند که فقط به مقصد نهایی فکر نکند، بلکه بداند قبل از رسیدن به مقصد باید همه‌ی این قطعه‌ها جمع‌آوری شوند.



پس مسیر خوب مسیری نیست که فقط کوتاه باشد؛ مسیر خوب باید مأموریت را کامل کند.

۴- Rifts یا شکاف‌های زمانی

Rift ها شکاف‌هایی هستند که در بافت زمان باز شده‌اند. اما هر شکاف را نمی‌توان بلافاصله بست. بعضی از آن‌ها فقط وقتی بسته می‌شوند که Chronos Agent به اندازه‌ی کافی Fragment جمع کرده باشد. در بازی، Rift ها پیش‌نیاز دارند. یعنی برای بستن بعضی شکاف‌ها باید تعداد مشخصی Fragment از قبل جمع شده باشد.



این باعث می‌شود مسئله فقط یک shortest path ساده نباشد. ممکن است یک Rift خیلی نزدیک باشد، اما هنوز اجازه‌ی بستن آن را نداشته باشید. بنابراین الگوریتم باید وضعیت مأموریت را هم در State ذخیره کند.

۵- دیوارها

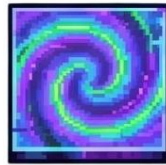
دیوارها خانه‌هایی هستند که قابل عبور نیستند. هیچ الگوریتمی نباید از آن‌ها عبور کند. اگر الگوریتمی مسیری پیدا کند که از دیوار رد می‌شود، یعنی آن الگوریتم درک درستی از دنیای بازی ندارد.



۶- Distortion Tiles یا خانه‌های اعوجاج زمانی

این خانه‌ها قابل عبورند، اما عبور از آن‌ها هزینه‌ی بیشتری دارد. در بازی، خانه‌های معمولی هزینه‌ی کم‌تری دارند، اما خانه‌های اعوجاج زمانی مسیر را سنگین‌تر می‌کنند. بنابراین الگوریتم‌هایی مثل

UCS و *A باید به هزینه‌ی مسیر توجه کنند، نه فقط تعداد قدم‌ها. هزینه‌های حرکت در دنیای Chronos Weaver شامل خانه‌ی معمولی، اعوجاج زمانی، Rift و صبر کردن هستند.



۷- Pads یا سکوهای پرش زمانی

بعضی خانه‌ها مثل آسانسور زمانی عمل می‌کنند. وقتی Chronos Agent وارد یکی از این خانه‌ها می‌شود، ممکن است به یک Pad دیگر منتقل شود. این یعنی موقع تولید همسایه‌ها، کافی نیست فقط خانه‌ی مجاور را حساب کنید؛ باید اثر خانه‌ی مقصد را هم اعمال کنید.



دشمنان زمانی

دشمنان موجوداتی هستند که در حلقه‌های زمانی مشخص حرکت می‌کنند. آن‌ها مثل نگهبانان خراب‌شده‌ی زمان‌اند؛ نه دقیقاً زنده‌اند، نه کاملاً مرده، فقط در مسیرهای تکراری خود حرکت می‌کنند.



Chronos Agent نباید وارد خانه‌ای شود که در لحظه‌ی بعد توسط دشمن اشغال می‌شود. همچنین اگر عامل و دشمن در یک لحظه جای خود را با هم عوض کنند، آن حرکت هم خطرناک است و باید غیرمجاز حساب شود. در کد بازی، دشمن‌ها مسیرهای چرخه‌ای دارند و موقع بررسی هر حرکت، فاز زمانی جدید هم در نظر گرفته می‌شود.

تعریف دقیق state

برای حل درست این مسئله، State فقط موقعیت x و y نیست. یک State در این بازی باید حداقل شامل موارد روبه‌رو باشد: $(x, y, collected_fragments, sealed_rifts, time_phase)$

یعنی دو State با موقعیت مکانی یکسان الزاما برابر نیستند. اگر در یکی از آن‌ها دو Fragment جمع شده باشد و در دیگری هیچ Fragment جمع نشده باشد، این دو وضعیت کاملا متفاوت‌اند. اگر در یکی زمان روی فاز ۳ باشد و در دیگری روی فاز ۷، موقعیت دشمنان ممکن است فرق کند.

در نتیجه، اگر فقط مختصات را در visited ذخیره کنید، الگوریتم شما ممکن است مسیر درست را حذف کند.

ماموریت شما

شما باید الگوریتم‌هایی را پیاده‌سازی کنید که بتوانند برای Chronos Agent یک برنامه‌ی حرکت پیدا کنند. برنامه‌ی حرکت یعنی دنباله‌ای از اکشن‌ها مثل: Left, Wait, Up, Right, Right, ...

این برنامه باید باعث شود که عامل:

۱- از نقطه‌ی شروع حرکت کند.

۲- از دیوارها عبور نکند.

۳- با دشمنان برخورد نکند.

۴- Fragmentها را جمع کند.

۵- Riftهای لازم را ببندد.

۶- در نهایت به Chronos Core برسد.

اگر الگوریتم موفق شود، بازی مسیر پیدا شده را روی نقشه نمایش می‌دهد و Chronos Agent آن را اجرا می‌کند. اگر موفق نشود، باید پیام مناسبی مثل No plan found یا Node limit reached برگرداند.

الگوریتم هایی که باید پیاده سازی شوند

۱. BFS — Breadth-First Search

BFS مثل یک کاوشگر صبور است. اول همه‌ی مسیرهای یک‌قدمی را بررسی می‌کند، بعد دوقدمی‌ها، بعد سه‌قدمی‌ها، و همین‌طور ادامه می‌دهد. در این پروژه، BFS باید با صف پیاده‌سازی شود. اگر همه‌ی حرکت‌ها هزینه‌ی یکسان داشتند، BFS می‌توانست کوتاه‌ترین مسیر از نظر تعداد قدم را پیدا کند. اما چون در این بازی خانه‌ها هزینه‌های متفاوت دارند، BFS الزاماً کم‌هزینه‌ترین مسیر را پیدا نمی‌کند.

از BFS انتظار داریم:

- ۱- کامل باشد، اگر فضای جست‌وجو قابل رسیدن باشد.
- ۲- مسیر را از طریق parent map بازسازی کند.
- ۳- visited را بر اساس State کامل نگه دارد، نه فقط مختصات.

۲. DFS — Depth-First Search

DFS مثل کسی است که یک راهرو را تا آخر می‌رود، حتی اگر کمی مشکوک باشد. این الگوریتم با stack پیاده‌سازی می‌شود.

DFS ممکن است سریع به یک جواب برسد، اما تضمینی ندارد که جوابش کوتاه یا کم‌هزینه باشد. در این پروژه DFS بیشتر برای مقایسه با بقیه‌ی الگوریتم‌هاست: گاهی سریع است، گاهی بی‌راهه می‌رود، و همین رفتار برای تحلیل الگوریتم‌ها جذاب است.

از DFS انتظار داریم:

- ۱- وارد حلقه‌ی بی‌نهایت نشود.
- ۲- State های دیده‌شده را ذخیره کند.
- ۳- اگر جواب پیدا کرد، مسیر معتبر برگرداند.

۳. UCS — Uniform Cost Search

UCS دیگر فقط تعداد قدم‌ها را نمی‌بیند؛ هزینه‌ی واقعی مسیر را حساب می‌کند. اگر عبور از Distortion Tile گران‌تر باشد، UCS باید این موضوع را بفهمد و شاید مسیر طولانی‌تر اما ارزان‌تر را انتخاب کند.

UCS باید با priority queue پیاده‌سازی شود. اولویت هر State برابر است با هزینه‌ی طی‌شده از شروع تا آن State.

از UCS انتظار داریم:

- ۱- کم‌هزینه‌ترین مسیر را پیدا کند، اگر هزینه‌ها نامنفی باشند.
- ۲- برای هر State بهترین g-cost شناخته‌شده را نگه دارد.
- ۳- اگر مسیر جدید به یک State ارزان‌تر بود، آن را به‌روزرسانی کند.

۴. Greedy Best-First Search

Greedy کمی عجول است. به جای اینکه کل هزینه‌ی مسیر را جدی بگیرد، بیشتر به این فکر می‌کند که از اینجا به بعد چقدر تا هدف مانده؟

Greedy با heuristic کار می‌کند. مثلاً می‌تواند تخمین بزند که عامل چقدر با Fragment‌های باقی‌مانده، Rift‌های باز، و Core فاصله دارد.

Greedy معمولاً سریع است، اما لزوماً بهترین جواب را نمی‌دهد. این الگوریتم برای دیدن قدرت و خطر heuristic‌ها عالی است.

۵. A* Search

A* ترکیبی از عقل و شهود است.

از یک طرف هزینه‌ی واقعی مسیر طی‌شده را دارد و از طرف دیگر تخمین هزینه‌ی باقی‌مانده را دارد:

پس اولویت آن می شود:

$$f(n) = g(n) + h(n)$$

A* اگر heuristic مناسبی داشته باشد، می تواند هم هوشمندانه جست و جو کند و هم مسیر بهینه پیدا کند. در گزارش پروژه باید توضیح دهید heuristic شما چیست و چرا برای این بازی مناسب است.

خروجی مورد انتظار الگوریتم ها

هر الگوریتم باید یک نتیجه ی قابل بررسی برگرداند. این نتیجه باید شامل موارد زیر باشد:

۱- algorithm name

۲- found / not found

۳- list of states in path

۴- list of actions

۵- total path cost

۶- number of visited states

۷- peak size of frontier

۸- runtime

۹- message

بازی از این اطلاعات برای نمایش مسیر، اجرای انیمیشن، و نمایش آمار الگوریتم استفاده می کند. در پنل بازی هم مواردی مثل نام الگوریتم، تعداد قدم ها، هزینه ی مسیر، تعداد State های دیده شده، بیشینه ی frontier و زمان اجرا نمایش داده می شود.

ضوابط ارزیابی

۱. تعریف الگوریتم‌های مرجع به عنوان **Baseline**: کدهای موجود در فایل `chronos_weaver.py` که در اختیار شما قرار گرفته است، صرفاً یک پیاده‌سازی اولیه و پایه محسوب می‌شوند و برای آشنایی شما قرار گرفته‌اند. شما موظف به ارائه‌ی یک پیاده‌سازی جدید، متفاوت و بازنویسی‌شده هستید که عملکرد بهینه‌تر و بهتری نسبت به پیاده‌سازی‌های پایه داشته باشد.
۲. الزام بهبود عملکرد و ارتقای معیارها: پیاده‌سازی جدید شما (به‌ویژه در الگوریتم A^*) باید به لطف طراحی یک تابع هیوریستیک جدید و هوشمند، بهبودی قطعی نسبت به مدل پایه نشان دهد. کد شما باید از نظر زمان اجرا (Runtime) و تعداد گره‌های بررسی‌شده (Visited States) بهینه‌تر از Baseline عمل کرده و در عین حال، بهینگی طول مسیر و هزینه نهایی را کاملاً حفظ کند.
۳. تضمین تعمیم‌پذیری: مدل پیشنهادی و پیاده‌سازی‌های شما نباید روی مپ پیش‌فرض بازی بایاس یا Overfit شده باشد. کدهای توسعه‌داده‌شده علاوه بر نقشه فعلی، بر روی تعدادی مپ جدید نیز سنجیده خواهد شد و حفظ پایداری، کارایی و تعمیم‌پذیری الگوریتم در محیط‌های جدید، مبنای سنجش عملکرد مدل خواهد بود.

بخش دوم: الگوریتم های جستجوی خصمانه

بعد از اینکه Chronos Agent بافت زمان را ترمیم کرد، مشکل تمام نمی شود. چون هر خط زمانی نجات یافته، سایه ای هم دارد. از دل یکی از Rift های نیمه بسته، موجودی بیرون می آید به نام **Chaos** یک ناهنجاری زمانی که نمی خواهد Core دوباره پایدار شود.



اینجا بازی وارد حالت دوم می شود: **Chronos Duel**.

در این بخش، مسئله دیگر فقط پیدا کردن مسیر نیست. حالا دو عامل داریم که به نوبت تصمیم می گیرند. Chronos می خواهد زنده بماند و Chaos را شکست دهد. Chaos هم می خواهد HP بازیکن را به صفر برساند. این بخش برای الگوریتم های جستجوی خصمانه طراحی شده است. در فایل **Duel**، اکشن هایی مثل حرکت، ضربه ی نزدیک، Beam، Mine و Charge تعریف شده اند.

اکشن های نبرد

در Chronos Duel هر بازیکن می تواند بسته به وضعیت، چند اکشن انجام دهد:

- حرکت به چهار جهت
- Strike وقتی دشمن مجاور است
- Beam وقتی دشمن در خط دید و برد مناسب است
- Mine وقتی انرژی کافی وجود دارد
- Wait یا Charge برای بازیابی انرژی

هدف اصلی این بخش این است که HP دشمن به صفر برسد. راهنمای درون بازی هم همین موارد را برای بازیکن توضیح می دهد: Strike دو واحد آسیب می زند، Beam برد ۴ دارد و انرژی مصرف می کند، Mine دو انرژی لازم دارد و هر کسی روی آن برود آسیب می بیند، و Charge وقتی انرژی پر نیست قابل انجام است.

الگوریتم هایی که باید پیاده سازی شوند

۱- Minimax

در Minimax فرض می کنیم هر دو طرف منطقی بازی می کنند. Chronos تلاش می کند بهترین نتیجه را بگیرد و Chaos تلاش می کند بدترین نتیجه را برای Chronos رقم بزند.

۲- Alpha-Beta Pruning

Alpha-Beta همان Minimax است، اما با حذف شاخه هایی که دیگر لازم نیست بررسی شوند. این الگوریتم باید همان تصمیم Minimax را بدهد، اما با State های کمتر.

۳- Expectimax

در Expectimax می توانیم فرض کنیم بعضی تصمیم ها کاملاً خصمانه نیستند یا رفتار طرف مقابل احتمالاتی است. این الگوریتم برای مقایسه با Minimax جالب است، چون نگاهش به حریف متفاوت است. در حالت Duel، بازی امکان انتخاب الگوریتم برای هر دو سمت را دارد: بازیکن Chronos و دشمن Chaos می توانند با Minimax، Alpha-Beta، یا Expectimax کنترل شوند. حالت Auto هم وجود دارد تا دو هوش مصنوعی مقابل هم بازی کنند.

ضوابط ارزیابی

۱. تعریف الگوریتم‌های مرجع به عنوان **Baseline**: کدهای موجود در فایل پایتون `chronos_duel.py` صرفاً یک پیاده‌سازی اولیه و **Baseline** محسوب می‌شوند. شما موظف به ارائه‌ی یک پیاده‌سازی جدید، متفاوت و بازنویسی شده هستید که در مواجهه با حریف، عملکرد و استراتژی بهینه‌تری نسبت به این پیاده‌سازی پایه داشته باشد.

۲. ارتقای تابع ارزیابی وضعیت (**evaluation_function**): بازنویسی پارامترها و وزن‌دهی‌های تابع ارزیابی پیش‌فرض الزامی است. تابع ارزیابی جدید باید متغیرهای پیچیده نظیر میزان HP، خط دید (Line of Sight)، فاصله تا حریف، مدیریت انرژی و موقعیت مین‌های کاشته‌شده را به صورت داینامیک فرموله کند.

۳. پیاده‌سازی مکانیزم مرتب‌سازی حرکت‌ها (**Action Ordering**): شما باید منطق آلفا-بتا را با اضافه کردن مکانیزم **Move Ordering** ارتقا دهید. کد جدید باید با اولویت‌بندی اکشن‌های مؤثرتر، تعداد گره‌های هرس شده را در عمق‌های بالاتر به طور معقولی نسبت به **Baseline** افزایش و همین‌طور زمان پاسخ‌دهی را کاهش دهد.

۴. تضمین تعمیم‌پذیری در نبرد خصمانه: عامل طراحی‌شده‌ی شما نباید صرفاً برای رفتار و مپ پیش‌فرض بازی تیون (Tune) شده باشد. کدهای توسعه داده‌شده روی چند مپ دیگر با چیدمان متفاوت و در برابر حریف‌هایی با رفتارهای جدید (تغییر الگوریتم حریف از **Minimax** به **Expectimax** یا تغییر سطح تهاجمی بودن و...) سنجیده خواهد شد و حفظ پایداری و کسب نرخ برد بالاتر نسبت به **Baseline**، مبنای سنجش این بخش خواهد بود.

گزارش پروژه

بعد از پیاده‌سازی، شما موظف به ارائه‌ی یک گزارش جامع هستید که علاوه بر تشریح دقیق ساختار کدها و روند پیاده‌سازی، باید به تجزیه و تحلیل خروجی‌ها پرداخته و معیارهای کلیدی زیر را به صورت دقیق مستند و بررسی کنید:

- فرمول ریاضی تابع هیوریستیک جدید خود را کامل تشریح کنید و قابل قبول بودن (Admissible) و سازگار بودن (Consistent) آن را به همراه اثر جریمه‌ی کاشی‌های مخرب اثبات نمایید.
- جدول مقایسه‌ای از معیارهای عددی شامل زمان اجرا بر حسب میلی ثانیه، تعداد گره‌های بررسی شده و هزینه نهایی مسیر برای پارت اول تهیه کنید و کارایی پیاده‌سازی جدید خود را نسبت به کد Baseline بسنجید.
- منطق پیاده‌سازی تابع ارزیابی جدید خود در پارت دوم را توضیح داده و نحوه وزن‌دهی به متغیرهای پویای محیط مانند میزان HP، مدیریت انرژی، خط دید و موقعیت مین‌ها را تحلیل کنید.
- کارایی الگوریتم آلفا-بتا را در عمق‌های بالاتر (حداقل تا عمق ۵) با و بدون اعمال مکانیزم مرتب‌سازی حرکت‌ها مقایسه کرده و تغییرات عددی تعداد گره‌های هرس شده و زمان پاسخ‌دهی را نسبت به عملکرد کد Baseline مقایسه و تجزیه تحلیل نمایید.
- عملکرد، امتیازات و نرخ برد عامل خود را روی نقشه‌های جدید با چیدمان متفاوت و در مواجهه با حریف‌هایی با رفتارهای جدید گزارش کنید و با مقایسه کارایی آن نسبت به مدل پایه، میزان تعمیم‌پذیری سیستم را تحلیل نمایید.

در ادامه، لازم است بر اساس نتایج آزمایش‌ها و داده‌های استخراج شده، به پرسش‌های تحلیلی زیر؛ پاسخ تشریحی و تحلیلی ارائه نمایید:

- کدام الگوریتم سریع‌تر جواب داد؟ کدام الگوریتم مسیر کم‌هزینه‌تری پیدا کرد؟
- BFS و UCS چه تفاوتی در این نقشه داشتند؟ اگر هزینه‌ی Distortion Tile را زیاده‌تر کنیم، رفتار الگوریتم‌ها چه تغییری می‌کند؟
- Greedy چه زمانی خوب عمل کرد و چه زمانی فریب خورد؟
- heuristic شما برای A^* چیست؟ آیا A^* همیشه بهتر از UCS بود؟ چرا؟

- اگر دشمن‌ها را حذف کنیم، مسئله چقدر ساده‌تر می‌شود؟
- چرا Chronos Duel یک مسئله‌ی جست‌وجوی خصمانه محسوب می‌شود و نقش دو عامل در آن چیست؟
- در Minimax، کدام عامل را Max و کدام را Min در نظر می‌گیرید؟ دلیل این انتخاب چیست؟
- تابع ارزیابی شما برای وضعیت‌های غیرنهایی چگونه طراحی شده است و چه عواملی مثل HP، انرژی، فاصله، خط دید یا مین‌ها در آن اثر دارند؟
- افزایش عمق جست‌وجو چه تأثیری بر کیفیت تصمیم‌ها، زمان اجرا و تعداد گره‌های بررسی شده دارد؟
- Alpha-Beta Pruning نسبت به Minimax چه مزیتی دارد و آیا همیشه همان حرکت Minimax را انتخاب می‌کند؟ ترتیب بررسی اکشن‌ها چه اثری روی کارایی Alpha-Beta و میزان هرس شدن شاخه‌ها دارد؟
- Expectimax چه تفاوتی با Minimax دارد و در چه شرایطی انتخاب مناسب‌تری است؟
- الگوریتم در چه موقعیت‌هایی باید بین حمله کردن، صبر کردن، ذخیره‌ی انرژی، استفاده از Beam یا گذاشتن Mine تصمیم بگیرد؟
- عامل طراحی شده‌ی شما بیشتر رفتار تهاجمی دارد یا دفاعی؟ این رفتار چه ارتباطی با تابع ارزیابی دارد؟

در مجموع، گزارش نهایی باید تصویری روشن از فرایند طراحی، توسعه و پیاده‌سازی پروژه، تجزیه و تحلیل عملکردها و مقایسه‌های مدنظر ارائه دهد، صرفاً ذکر نتایج عددی و... کافی نیست و باید به طور کامل و جامع، تجزیه و تحلیل نمائید.